SMT Solvers and DPLL($\mathcal{T}$)

Watching theory solving, one conflict at a time

The problem: is this formula satisfiable?

Is there a pair of integers (x, y) such that all of these hold?

$$x \geq 1, \qquad y \geq 1, \qquad x + y \leq 5,$$
$$(x \geq 4 \vee y \geq 4)$$

Satisfiable? Predict: `sat` or `unsat`?

Over the next slides we watch an SMT solver decide this, one step at a time. You will encode it in `Demo/demo.py` afterwards.

Step 0: the encoding, and what an atom means

1. Name the atoms

$p_1 \quad :\equiv x \geq 1$

$p_2 \quad :\equiv y \geq 1$

$p_3 \quad :\equiv x + y \leq 5$

$p_4 \quad :\equiv x \geq 4$

$p_5 \quad :\equiv y \geq 4$

2. Write clauses

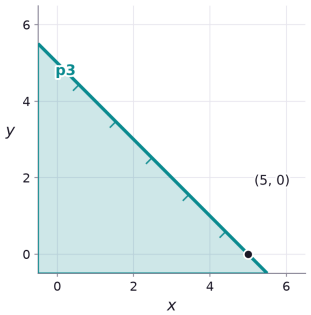
$C_1 : \quad p_1$

$C_2 : \quad p_2$

$C_3 : \quad p_3$

$C_4 : \quad p_4 \vee p_5$

The SAT engine sees only these Boolean clauses.



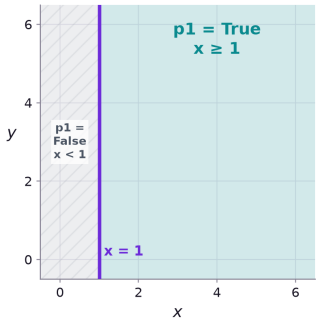
$p_3 :\equiv x + y \leq 5$ is the line $x + y = 5$. True keeps the shaded side.

A **half-plane** is one side of a line. Every atom is a line in the (x, y) plane: setting it True keeps one half-plane, False keeps the other. The **theory solver** knows this meaning and checks that the chosen halves can overlap.

Step 1: an atom cuts the plane in half

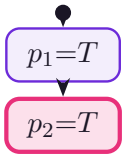
$p_1{=}T$ unit clause C_1

The SAT engine sets $p_1 = \text{True}$ ($p_1 :\equiv x \geq 1$).

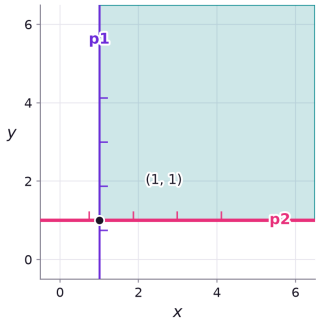


The atom $x \geq 1$ is a line. True keeps the right half, False would keep the left half.

Step 2: a second atom cuts again

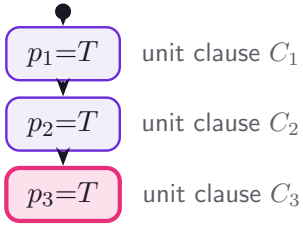


$p_2 =$ True ($p_2 :\equiv y \geq 1$).

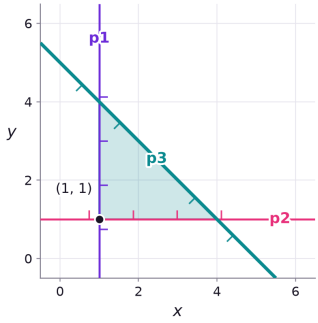


Both literals must hold, so we keep the *overlap* of the two half-planes: a quadrant.

Step 3: the region is a triangle



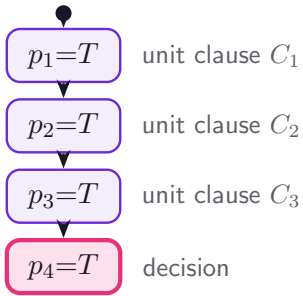
$p_3 =$ True ($p_3 :\equiv x + y \leq 5$).



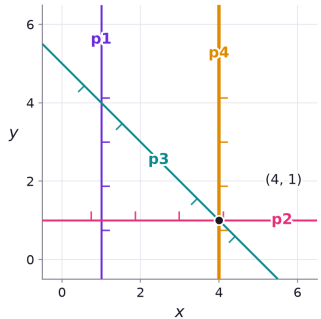
Three half-planes overlap in a triangle. The dot is a point Z3 picks inside it.

Consistent = the half-planes of all assigned literals have a common point.

Step 4: a guess shrinks the region

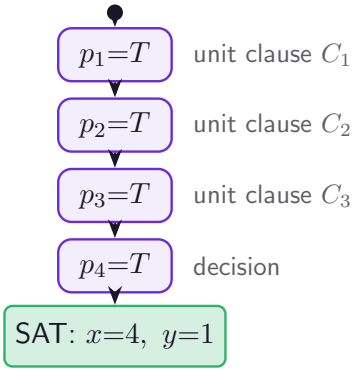


Clause $p_4 \vee p_5$ needs one of them. The SAT engine **guesses** $p_4 = \text{True}$ ($x \geq 4$).

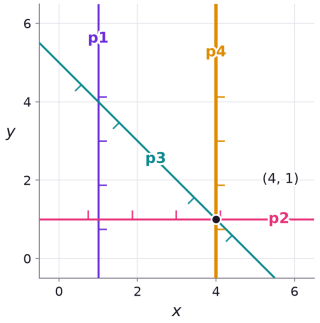


Adding $x \geq 4$ squeezes the triangle to the single point $(4, 1)$: still consistent.

The run ends: SAT



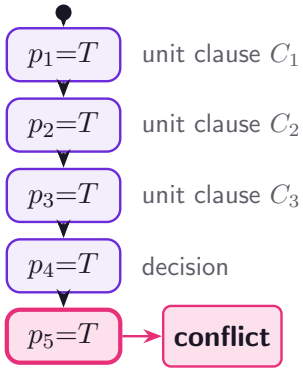
Every clause holds and the theory check is consistent.



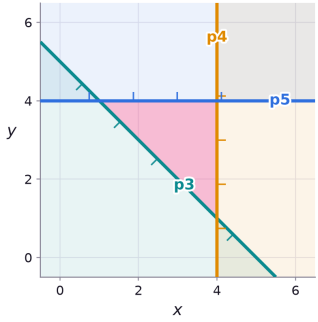
The witness $(4, 1)$ really works: $4 \geq 1,\ 1 \geq 1,$ $4 + 1 \leq 5,\ 4 \geq 4.$

The formula is **satisfiable**: $x = 4,\ y = 1.$

What if the solver also guessed $y \geq 4$?



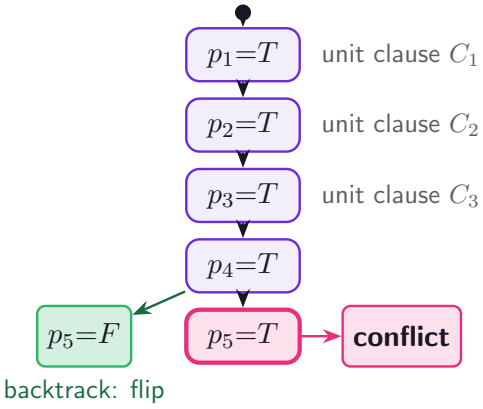
Suppose it also sets $p_5 = \text{True}$ ($y \geq 4$).



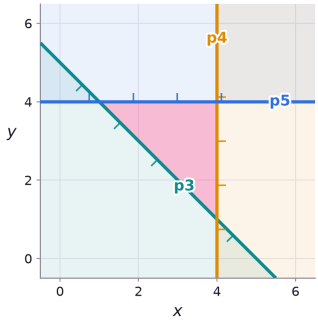
Only a pink gap is left: no point lies in all three half-planes.

Consistent: some point satisfies all assigned atoms (overlap).
Conflict: no point does (empty overlap), and the solver names the atoms that clash.

A conflict becomes a learned clause

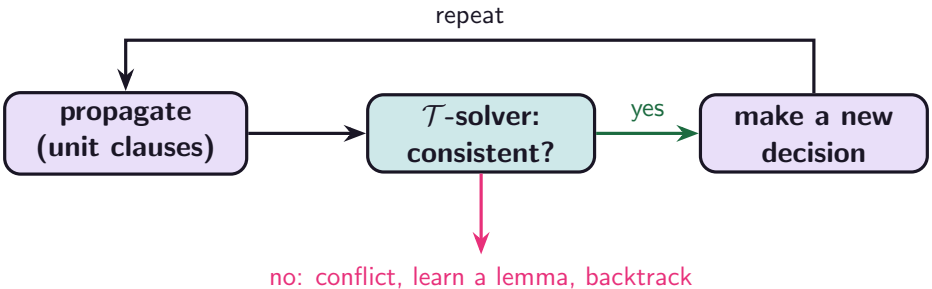


Learned: $\neg p_4 \vee \neg p_5$, the negation of the two decisions. Then **backtrack**: flip p_5.



The same three lines. p_1 and p_2 were not needed to explain the conflict. The lemma is never forgotten.

When does the theory solver run?



The $\mathcal{T}$-solver is called **just before every new decision**, on the partial assignment so far, not only at a leaf where every variable is assigned. A conflict found early cuts off every guess that would have come after it.

DPLL($\mathcal{T}$) vs. CDCL($\mathcal{T}$)

DPLL($\mathcal{T}$)

Guess, propagate, check the theory.
On a conflict it **backtracks** and tries the other value.
Remembers **nothing**: the same bad combination can be tried again.

CDCL($\mathcal{T}$)

Same loop, but on a conflict it **learns a clause** (a learned clause) and may **backjump** several levels.
The same mistake is never repeated.

In our run the conflict gave the clause $\neg p_4 \vee \neg p_5$, the negation of the two decisions: from then on the SAT engine never again sets $x \geq 4$ and $y \geq 4$ together.

DPLL($\mathcal{T}$) is the traditional name of the framework. Modern solvers, Z3 included, run CDCL inside it, so you will see both names. Our tracer learns clauses but backtracks one level at a time.

Now let's watch
the solver think.

The challenge: find a point (x, y)

Inside the island

$x \geq 0$

$y \geq 0$

$x + y \leq 8$

Clue A: east of 3, or far north

$x \geq 3 \vee y \geq 6$

Clue B: west of 2, or far north

$x \leq 2 \vee y \geq 6$

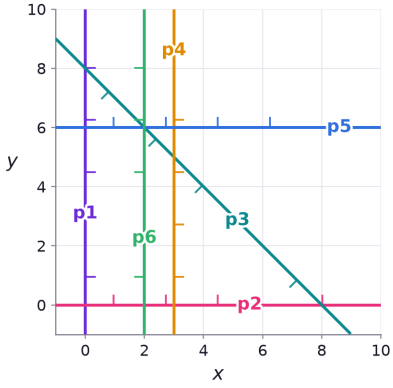
Run it in the terminal:

```
python3 Demo/dpllt_trace.py "x >= 0" "y >= 0" "x
   + y <= 8" "Or(x >= 3, y >= 6)" "Or(x <= 2, y
  >= 6)" --reals x y
```

Keep the terminal open. Every step on the next slides is a step of this run.

Step 0: the propositional encoding

```
Each theory atom becomes a Boolean variable
  the SAT engine can guess:
    p1  :=  x >= 0
    p2  :=  y >= 0
    p3  :=  x + y <= 8
    p4  :=  x >= 3
    p5  :=  y >= 6
    p6  :=  x <= 2
The formula becomes a set of clauses over
  those variables:
    C1:  p1
    C2:  p2
    C3:  p3
    C4:  p4 ∨ p5
    C5:  p6 ∨ p5
```



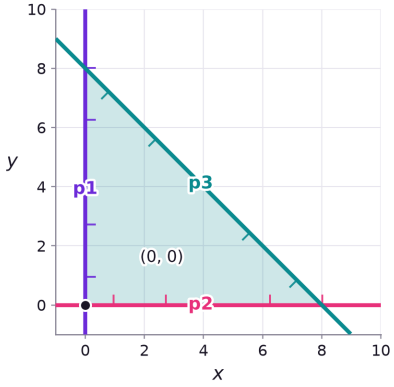
p_1: $x \geq 0$
p_2: $y \geq 0$
p_3: $x + y \leq 8$
p_4: $x \geq 3$
p_5: $y \geq 6$
p_6: $x \leq 2$

Each atom p_i is a line. Its colour is the same in every picture.

Step 1: unit clauses → consistent

```
Level 0 (no decisions yet)
propagate  p1 (x >= 0) = True    [C1: p1 -- it
   is a unit clause]
propagate  p2 (y >= 0) = True    [C2: p2 -- it
   is a unit clause]
propagate  p3 (x + y <= 8) = True    [C3: p3
  -- it is a unit clause]
check      Z3 consistency check #1
theory     consistent
```

- p_1: $x \geq 0$ *(new)*
- p_2: $y \geq 0$ *(new)*
- p_3: $x + y \leq 8$ *(new)*

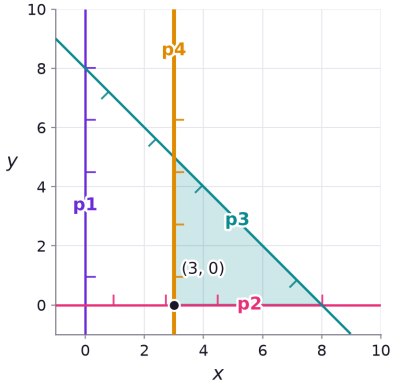


Shaded: all assigned literals hold. Thick line: newest literal. Dot: Z3's point $(0,\ 0)$.

Step 2: guess p4 = True → consistent

```
Level 1: decide  p4 (x >= 3) = True   (a
  guess)
check       Z3 consistency check #2
theory      consistent
```

- p_1: $x \geq 0$
- p_2: $y \geq 0$
- p_3: $x + y \leq 8$
- p_4: $x \geq 3$ *(new)*

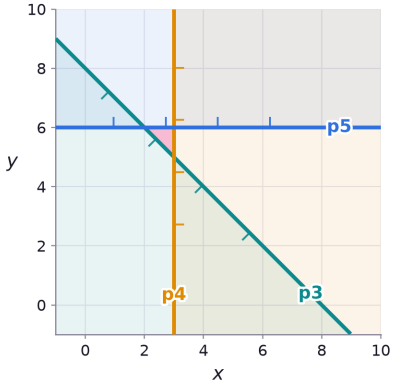


Shaded: all assigned literals hold. Thick line: newest literal. Dot: Z3's point $(3,\ 0)$.

Step 3: guess p5 = True → CONFLICT

```
Level 2: decide  p5 (y >= 6) = True   (a
  guess)
check       Z3 consistency check #3
theory      CONFLICT: x + y <= 8, x >= 3, y >=
   6 cannot all hold
learn       C6: ¬p4 ∨ ¬p5   (negation of the
  decisions)
```

- p_3: $x + y \leq 8$
- p_4: $x \geq 3$
- p_5: $y \geq 6$ *(new)*
- pink: breaks every literal



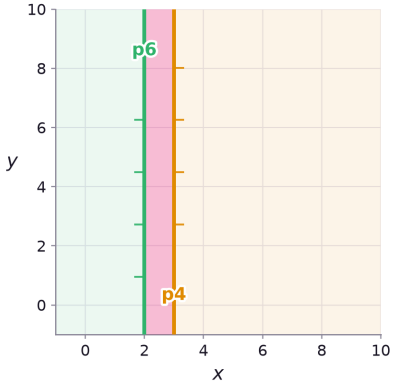
Learned: $\neg p_4 \vee \neg p_5$ (negation of the decisions). No point satisfies $x + y \leq 8$, $x \geq 3$, $y \geq 6$ together.

Step 4: flip p5 = False → CONFLICT

```
backtrack  undo decision p5 = True and
  everything after it

Level 2: flip p5 = False  (try the other
  value)
propagate  p6 (x <= 2) = True    [C5: p6 ∨ p5
  -- all other literals are False]
check      Z3 consistency check #4
theory     CONFLICT: x >= 3, x <= 2 cannot
  all hold
learn      C7: ¬p4   (negation of the
  decisions)
```

- p_4: $x \geq 3$
- p_6: $x \leq 2$ *(new)*
- pink: breaks every literal



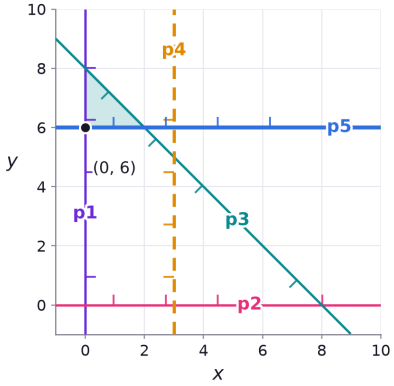
Learned: $\neg p_4$ (the only open decision). No point satisfies $x \geq 3$, $x \leq 2$ together.

Step 5: flip p4 = False → consistent → SAT

```
backtrack  undo decision p4 = True and
  everything after it

Level 1: flip p4 = False  (try the other
  value)
propagate  p5 (y >= 6) = True    [C4: p4 ∨ p5
  -- all other literals are False]
check      Z3 consistency check #5
theory     consistent
result     every clause is satisfied  ->  SAT
```

p_1: $x \geq 0$
p_2: $y \geq 0$
p_3: $x + y \leq 8$
p_4: $x < 3$ *(new)*
p_5: $y \geq 6$ *(new)*



Shaded: all assigned literals hold. Thick line: newest literal. Dot: Z3's point $(0,\ 6)$.

What the run taught the SAT engine

Step	Learned clause	Half-planes	Picture
3	$\neg p_4 \vee \neg p_5$	$x + y \leq 8,\ x \geq 3,\ y \geq 6$	three half-planes, empty triangle gap
4	$\neg p_4$	$x \geq 3,\ x \leq 2$	two half-planes, no overlap

Result: **SAT** with $x = 0,\ y = 6$ (decisions: 2, conflicts: 2, Z3 consistency checks: 5).

Every conflict was a geometric fact: some half-planes have an empty intersection. The SAT engine learns the negation of its decisions, so it never repeats that combination.

Theories you can plug in

Theory	Example atom	How it is decided
Linear arithmetic	$x + y \leq 8$	simplex (integers: add branching)
Nonlinear arithmetic	$x \cdot y > 2$	reals decidable, integers not
Arrays	$a[i] = v$	read-over-write rules
Bit-vectors	$x \,\&\, (x - 1) = 0$	bit-blasting into SAT

Also available: datatypes, floating point, sequences. A formula can mix theories because the theory solvers exchange the equalities they discover, with the SAT engine coordinating.

Your turn: linear arithmetic in Z3

Find integers x, y and a real r such that

1. $x + 2y = 12$, x is even, $y > 0$
2. $x \neq y$ and $|x - y| \leq 3$
3. $\max(x, y) = 5$
4. $r = x/3$ using *real* division
5. $r < 1$ or $r > 2$

Print r exactly, as a decimal, rounded down, and check whether it is a whole number.

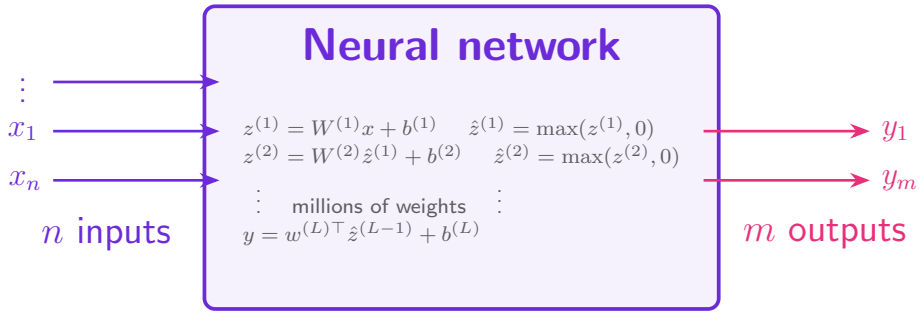
Then prove: for every integer n, if n is even then $n + 2$ is even.

Try it yourself first! All the syntax you need is in **LRA_LIA/syntax.pdf**.

Write your solution in `LRA_LIA/demo_template.py`. The TAs will come around to help.

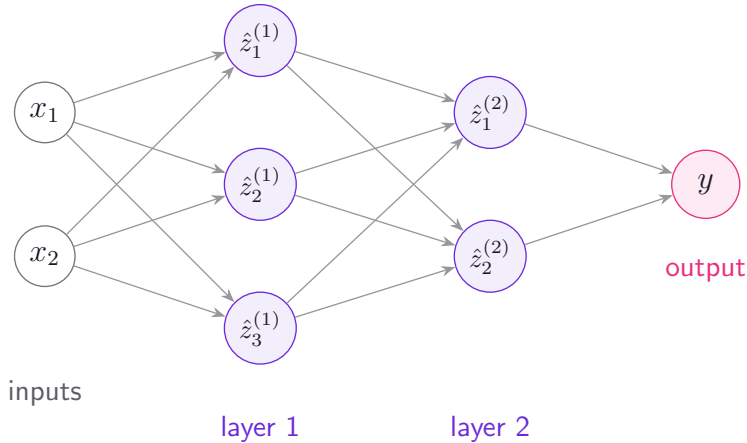
NEURAL NETWORK VERIFICATION

A neural network: a black box full of algebra



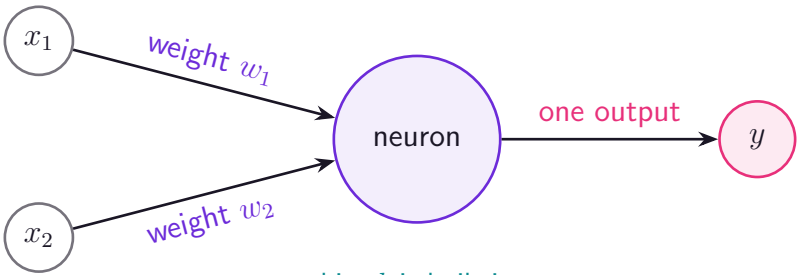
This is what runs your lovely ChatGPT/Gemini.

Inside: many small units called neurons



Each circle is a **neuron**. A layer's outputs are the next layer's inputs.

One neuron: inputs in, one number out



The neuron takes a few numbers, mixes them with its fixed constants w_1, w_2, b, and returns a single number.

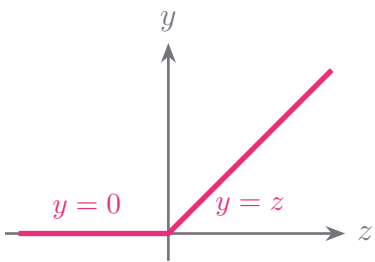
The math behind a neuron

Step 1: weighted sum

$$z = w_1 x_1 + w_2 x_2 + b$$

Step 2: ReLU

$$y = \max(z, 0) = \begin{cases} z & z \geq 0 \\ 0 & z < 0 \end{cases}$$



the ReLU function

In Z3: `y == If(z >= 0, z, 0)`

Our neuron, and the inputs it is allowed to see

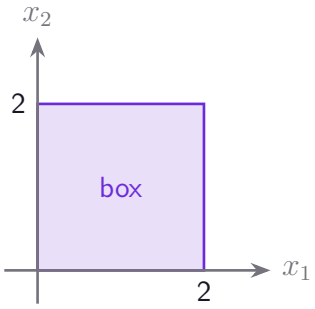
The neuron

$$z = 2x_1 - x_2 - 1$$

$$y = \max(z, 0)$$

The box (allowed inputs)

$$0 \le x_1 \le 2, \qquad 0 \le x_2 \le 2$$



Verification asks questions about *every* point in the box, not just one.

Task A: can the neuron output at least 3?

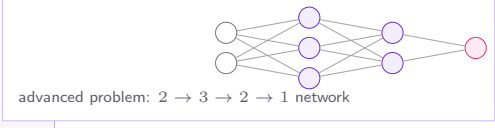
Problem. Is there an input in the box for which the neuron outputs $y \geq 3$?

Neuron	Box	Query
$z = 2x_1 - x_2 - 1$	$0 \leq x_1 \leq 2$	$y \geq 3$
$y = \max(z, 0)$	$0 \leq x_2 \leq 2$	

Put the neuron, the box and the query into one solver and call `check()`. Print (x_1, x_2, y) if `sat`.

Running your tasks

```
python3 neuron_template.py --A
python3 neuron_template.py --B --C
python3 neuron_template.py  (all tasks)
```



Task B: can it output at least 3.5?

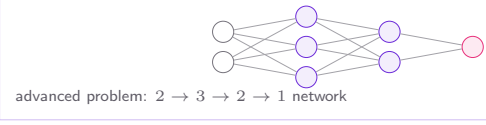
Problem. Same neuron and box as Task A. Is there an input in the box with $y \geq 3.5$?

Query

$$y \geq 3.5$$

- If `unsat`: you have *proved* that $y < 3.5$ for every input in the box.
- What is the largest value y can take on the box? Try other thresholds to find out.

```
python3 neuron_template.py --B
```



Task C: the region where the neuron is silent

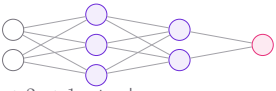
Problem. Find an input in the box where both inputs are positive but the neuron outputs nothing.

Query

$$x_1 > 0, \qquad x_2 > 0, \qquad y = 0$$

Print the input you find. Which part of the box is this?

```
python3 neuron_template.py --C
```



advanced problem: $2 \to 3 \to 2 \to 1$ network

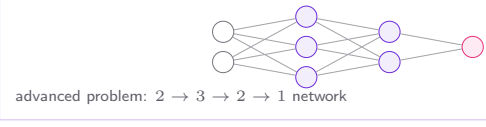
Task D (optional): a ReLU without `If`

Problem. Encode the neuron using a $0/1$ variable p instead of `If`. For bounds $l \le z \le u$:

$$y \ge 0, \quad y \ge z, \quad y \le z - l(1-p), \quad y \le u\,p, \quad p \in \{0,1\}$$

- Work out l and u for z over the box by hand.
- Redo Tasks A and B with it. The answers must not change.
- Repeat with $l = -100000$, $u = 100000$.

```
python3 neuron_template.py --D
```

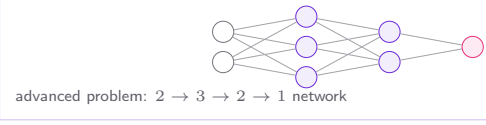


Task E (optional): range of z

Problem. Compute the smallest and the largest value of $z = 2x_1 - x_2 - 1$ over the box.

A neuron is **stable** if z never changes sign on the box, and **unstable** otherwise. Which is it here?

```
python3 neuron_template.py --E
```



Task F (optional): sensitivity

Problem. Let $c = (1, \frac{1}{2})$ and let y_0 be the neuron's output at c. For a radius ε and a threshold δ:

$$\exists\, x' \in \text{box}: \ |x'_1 - 1| \le \varepsilon, \ |x'_2 - \tfrac{1}{2}| \le \varepsilon, \ |y(x') - y_0| \ge \delta \ ?$$

Try $\varepsilon \in \{\frac{1}{4}, \frac{1}{2}, 1\}$ and $\delta \in \{\frac{1}{2}, 1, 2\}$. If `sat`, print x'.

```
python3 neuron_template.py --F
```



advanced problem: $2 \to 3 \to 2 \to 1$ network

Theory of Bit Vectors

What is the theory of bit-vectors?

- A bit-vector of width n is a tuple $(b_{n-1}, \ldots, b_0) \in \{0,1\}^n$.
- Universe $\{0,1\}^n$. Symbols: $\&$, $|$, $\oplus$, $+$, $\leq_u$, $\ldots$
  - bitwise operations act on each bit
  - arithmetic is **mod** 2^n, reading the tuple as a number
- Used for hardware, crypto and systems verification.
- Decided by **bit-blasting**: turn the formula into an equisatisfiable propositional formula, then run CDCL.

A 4-bit register $w = 1100$ (12). Doubling gives 11000, truncated to 4 bits: 1000 (8). So $w + w <_u w$ is satisfiable: **overflow**.

Reading the hex values in the demo

Hex is a short way to write bits: the prefix `0x` means hexadecimal, and each hex digit is exactly 4 bits (`A`=10, `B`=11, …, `F`=15).

Hex	Bits	Decimal	What it does in the demo
`0x0F`	0000 1111	15	a *mask*: its 1-bits select the low 4 bits
¬`0x0F`	1111 0000	240	¬ flips every bit; XOR flips the high 4 bits
`0xFD`	1111 1101	253	the target value of condition 1
`0x0A`, `0x0B`	1010, 1011	10, 11	$\gg 4$ drops a hex digit: $x{+}100$ starts with `A`/`B`
`0x20`	0010 0000	32	first printable ASCII character: the space
`0x7E`	0111 1110	126	last printable ASCII character: ~

`0xFD` and 253 are the same bit-vector to the solver. Hex only makes bit patterns easier to read.

Your turn: encode it in demo.py

Find an 8-bit value x such that

1. $(x + 200) \oplus \neg\texttt{0x0F} = \texttt{0xFD}$ (8-bit arithmetic, wraps mod 256)
2. x zero-extended to 16 bits, plus 100, shifted right (logical) by 4, is `0x0A` or `0x0B`
3. x is a printable ASCII character (`0x20`–`0x7E`)

Fill in the `#TODO` in `bitvec/demo.py`.
All the syntax you need is in **syntax.pdf**.

Hex values: see the previous slide. Take **5–10 minutes**. The TAs will come around to help.

Expected output

```
sat
[x = 69]
```

$x = 69 = 0x45$, the character `'E'`. It is the only solution.

The problem: reverse a hash

FNV-style hash of a string $b_1 \dots b_n$ (32-bit state):

$$s_0 = \texttt{0x811C9DC5}, \quad s_i = (s_{i-1} \oplus b_i) \cdot \texttt{0x01000193} \bmod 2^{32}, \quad h = s_n \oplus (s_n \gg 15)$$

Fill in `hash_template.py`:

1. **Preimage:** given a digest, find a printable string of length 8 that hashes to it.
2. **Collision:** find two *different* printable strings of length 8 with the same digest.

Hints

1. Each character is 8 bits but the state is 32: widen it first.
2. The final $\gg$ is a *logical* shift. In Z3, >> is not.
3. Printable means $\texttt{0x20} \le b \le \texttt{0x7E}$ as *unsigned*; "different" means some position differs.

What to remember

- An SMT solver is a **SAT engine plus theory solvers**.
- A linear-arithmetic literal is a **half-plane**; a theory check asks whether they overlap.
- A **conflict** is an empty intersection; the smallest conflicting set becomes a **lemma** the SAT engine never forgets.

Next: what if the formula says "for every x"? Quantifiers go beyond DPLL($\mathcal{T}$).

The tracer is a simplified textbook DPLL($\mathcal{T}$). Real Z3 backjumps, learns stronger lemmas and uses the simplex method instead of drawing, so its steps differ but its answers agree.